Slim-Mamba: An Efficient and Reconfigurable Mamba-Based Accelerator for Massive MIMO Indoor Localization

IEEE Publication Technology, *Staff, IEEE,*

***Abstract*—Massive MIMO radio measurements provide rich spatial and delay-domain information for indoor localization, but their high-dimensional channel features make real-time neural inference challenging on resource-constrained hardware. This paper presents Slim-Mamba, an efficient and reconfigurable Mamba-based accelerator for Massive MIMO indoor localization. At the algorithm level, Slim-Mamba simplifies the original Mamba-style selective state-space block into a lightweight gated recurrent update, preserving temporal state modelling and output gating while reducing the complexity of the selective-scan computation. The deployed model contains 816k trainable parameters and reduces the model size to 3.40 MB while maintaining competitive localization accuracy. At the architecture level, the accelerator supports reconfigurable inter-block execution, MAC-fabric scheduling, and quantization. The Slim-Mamba blocks can operate with either streaming data transfer or preload execution, while a shared $4 \times 4 \times 4$ MAC fabric is reused across multiple projection operators through configurable scheduling. In addition, a unified requantization engine enables several modes for scaling, rounding, and saturation. A fine-grained pipeline further overlaps projection, nonlinear activation, state update, and output gating, reducing latency by 23.8%. Combined with the reconfigurable architecture, the design achieves both efficient resource utilization and lower inference delay. Implemented on a Zynq MPSoC FPGA, the accelerator achieves a mean localization error of 0.14103 m, close to the float result of 0.13500 m, and reaches 471 frames/s at 100 MHz. The design uses 22.21% LUTs, 14.11% FFs, 30.59% BRAMs, and 19.68% DSPs, while the programmable logic (PL) design consumes only 0.94 W, demonstrating efficient real-time deployment of Mamba-style localization models on FPGA.**

***Index Terms*—Massive MIMO indoor localization, Mamba, FPGA accelerator, quantization, reconfigurable architecture, edge AI.**

I. Introduction

INDOOR localization is becoming an important function in future wireless systems, where communication, sensing, and positioning are expected to be more tightly integrated. In the context of 6G, localization has been identified as both a key service and an enabling technology for location-aware applications such as robotic navigation, emergency healthcare, smart environments, and intelligent transportation [1]. However, accurate indoor positioning remains challenging because indoor radio propagation is affected by multipath effects, noise, and the surrounding radio environment [2].

Massive multiple-input multiple-output (MIMO) systems provide a promising measurement platform for radio-based indoor localization. The LuViRA dataset used in this paper provides synchronized vision, radio, audio, and motion-capture measurements for indoor localization research [3], [4]. In its radio modality, the channel response is measured using the LuMaMi Massive MIMO testbed [5], enabling the collection of rich Massive MIMO channel information that can be used for learning-based localization, making LuViRA a suitable dataset for evaluating radio-based localization models under practical indoor propagation conditions. Previous FCNN-based Massive MIMO localization studies used spatial covariance matrices and truncated CIRs as fingerprints, and reached centimetre-level positioning accuracy [6]. However, the resulting FCNN models require network sizes ranging from hundreds of megabytes to more than 1 GB. The extension in [7] further confirms that increasing network size is a central challenge when ML-based localization is applied to Massive MIMO systems, which may hinder the training and fine-tuning of the network. Therefore, it is essential to develop efficient localization algorithms.

Mamba [8], a SSM(state-space models)-based model, provide a promising direction for this purpose. It performs recurrent state updates with input-dependent parameters while maintaining linear complexity with respect to sequence length. This characteristic is attractive for indoor localization because radio measurements can be represented as short sequences of channel features. However, the original Mamba architecture was not designed for Massive MIMO localization. To make it better suited for this task, in this paper, we explored Mamba-style architectures and simplified the original Mamba architecture to our Slim-Mamba model.

Recent studies have further shown that Mamba-style models require dedicated hardware-aware optimization when deployed on FPGA. Existing Mamba accelerators have explored computation reordering, accurate quantization and pipelined execution dataflow, and hybrid dataflows [9]–[11]. These works suggest that the efficiency of Mamba acceleration depends on reusable hardware structures that maximize resource sharing, efficient pipelined dataflows that improve processing throughput and reduce inter-stage stalls, and quantization strategies that balance computational efficiency with inference accuracy. Motivated by this observation, this paper explored hardware realization of Slim-Mamba for Massive MIMO indoor localization. The proposed reconfigurable accelerator reuses computation resources across different projection layers, supports configurable inter-block execution, applies a unified and configurable requantization module for different layers, and uses fine-grained FIFO-based pipelining to reduce waiting time between projection, nonlinear activation, state update, and output gating.

This paper was produced by the IEEE Publication Technology Group. They are in Piscataway, NJ.

Manuscript received April 19, 2021; revised August 16, 2021.

II. Mamba-Based Model Exploration for Localization

A. Localization Task and Sequence Input

The target task is radio-based indoor localization on the LuViRA dataset, where each sample is represented by a short sequence of Massive MIMO channel observations. In the deployed setting, each observation contains 2100 input features derived from the CIR and power representation used consistently in training, software evaluation, and hardware export. For one inference window, the model receives $\mathbf{X} = [\mathbf{x}_{t-K+1}, \ldots, \mathbf{x}_t]$, where $\mathbf{x}_i \in \mathbb{R}^{D_{\text{in}}}$ and $D_{\text{in}} = 2100$, and predicts the two-dimensional user position. In this task, the useful temporal information is concentrated in short-range channel variation rather than in long-range sequence dependency.

This property guides the model exploration. The original Mamba block is attractive because it combines recurrent state propagation with input-dependent modulation, but its full selective state-space parameterization was introduced for more general sequence modeling problems. For the present localization setting, the main requirement is to retain short-window temporal tracking while keeping the operator set regular enough for fixed-point FPGA mapping. Accordingly, the model exploration in this work seeks the smallest Mamba-like recurrent structure that preserves the localization accuracy required for deployment.

B. From Original Mamba to Slim-Mamba

The exploration considered two reference directions. The first was a Vanilla Mamba v1 implementation following the original selective state-space block. The second was a channel-aware variant with additional feature-fusion branches for stronger cross-channel interaction modeling. The channel-aware variant achieved the best software-only accuracy when augmented input features were available, but at the cost of substantially larger model size, longer training time, and a more complex hardware mapping. In contrast, the Vanilla Mamba baseline showed that the full selective-scan path was not necessary to capture the short-window localization dynamics in LuViRA. These observations motivated the final Slim-Mamba design.

Slim-Mamba follows a task-driven simplification principle. It retains the front-end and back-end skeleton of the original Mamba block, including normalization, input projection and split, output gating, output projection, and residual addition. The main modification is confined to the state path, so Table I lists only the operators that are replaced. In the original Mamba block, the state branch generates input-dependent Δ_t, B_t, and C_t, discretizes the continuous-time parameters (A, B), and then applies a selective scan recurrence. Slim-Mamba replaces this sequence with a direct discrete-time gated update that is easier to quantize and implement in hardware.

Figure 1 summarizes the replacement. The state branch first applies depthwise convolution and SiLU to produce the state-branch feature $\tilde{\mathbf{x}}_t^{(s)}$. A single input-dependent update coefficient is then generated as

$$\lambda_t = \sigma\left(W_\lambda * \tilde{\mathbf{x}}_t^{(s)} + b_\lambda\right), \tag{1}$$

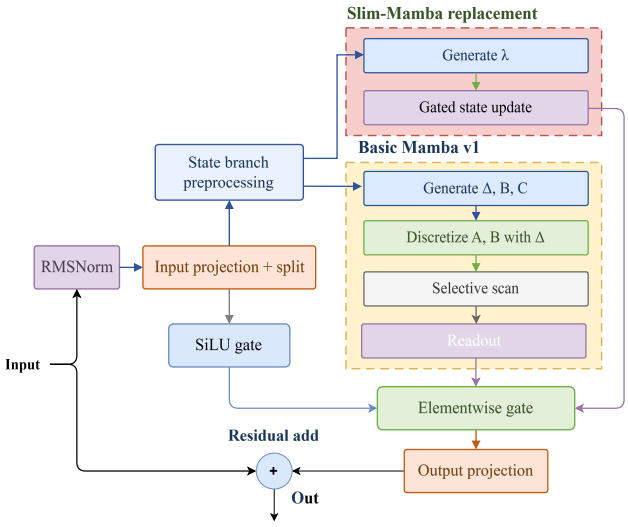

Fig. 1. Block-level replacement from the original Mamba state path to the Slim-Mamba state path.

TABLE I
Main operator substitutions from original Mamba to Slim-Mamba.

Stage	Original Mamba	Slim-Mamba
Local mixing	Conv + SiLU	DWConv + SiLU
Parameter generation	B_t, C_t, Δ_t, then discretize(Δ_t, A, B)	Single update gate λ_t
State update	$\mathbf{h}_t = \bar{A}_t \mathbf{h}_{t-1} + \bar{B}_t \tilde{\mathbf{x}}_t^{(s)}$	$\mathbf{h}_t = \lambda_t \odot \mathbf{h}_{t-1} + (1-\lambda_t) \odot \tilde{\mathbf{x}}_t^{(s)}$
Readout	$\mathbf{r}_t = C_t \mathbf{h}_t + D\tilde{\mathbf{x}}_t^{(s)}$	Direct state readout $\mathbf{r}_t = \mathbf{h}_t$

where $\sigma(\cdot)$ denotes the sigmoid function. Instead of discretizing a continuous-time state-space model, Slim-Mamba updates the recurrent state directly as

$$\mathbf{h}_t = \lambda_t \odot \mathbf{h}_{t-1} + (1 - \lambda_t) \odot \tilde{\mathbf{x}}_t^{(s)}. \tag{2}$$

The updated state is then modulated by the gate branch

$$\mathbf{g}_t = \text{SiLU}(\mathbf{x}_t^{(g)}), \tag{3}$$

and mapped to the block output through

$$\mathbf{y}_t = W_{\text{out}}(\mathbf{h}_t \odot \mathbf{g}_t). \tag{4}$$

Compared with the original Mamba block, this replacement removes three major sources of hardware complexity: time-varying generation of multiple SSM parameters, the discretization step from Δ_t to $(\bar{A}_t, \bar{B}_t)$, and the explicit (C_t, D)-based readout. The resulting model remains recurrent and input-dependent, but its operator set is reduced to projection, element-wise nonlinearities, gated state update, and output modulation. This reduced operator set is the main reason why Slim-Mamba is better suited to shared MAC reuse, fixed-point quantization, and fine-grained pipelining.

C. Deployed Configuration

The deployed Slim-Mamba configuration uses $D_{\text{in}} = 2100$, $K = 2$, proj_dim $= 64$, $d_{\text{model}} = 128$, $n_{\text{layer}} = 4$, patch length $P = 2$, and stride $S = 2$. This setting was selected

TABLE II
TRAINABLE PARAMETER BREAKDOWN OF THE DEPLOYED SLIM-MAMBA CONFIGURATION.

Component	Parameters
Input projection $D_{\text{in}} \rightarrow$ proj_dim	134,464
Patch embedding Conv1d	16,512
Four block RMSNorm layers	512
Four block in-proj layers	262,144
Four block update-gate projection layers	263,168
Four block out-proj layers	131,072
Final RMSNorm	128
Flat output layer	8,256
Regression head proj_dim $\rightarrow$ 2	130
Total	**816,386**

because it preserves the compact short-window behavior observed in software while keeping the projection and state-update dimensions aligned with the intended FPGA datapath. The final model contains 816,386 trainable parameters and occupies 3.40 MB.

This parameter distribution is also relevant from the hardware perspective. Most trainable weights are concentrated in the repeated input projection, update-gate projection, and output projection paths. These are exactly the stages later mapped to the shared reconfigurable MAC fabric. Therefore, the algorithmic simplification and the hardware architecture are aligned not only at the level of recurrence definition, but also at the level of parameter concentration and dataflow structure.

III. QUANTIZATION WORKFLOW AND HARDWARE IMPLEMENTATION

The Slim-Mamba accelerator is implemented with fixed-point arithmetic to reduce memory cost, DSP usage, and data-movement overhead on FPGA. The rounding, scaling, and fixed-point data format must also be considered to make sure the accuracy is close between the software model and the RTL datapath. For this reason, we use a three-level quantization tool, moving from a floating-point Python reference, to a bit-true C++ fixed-point model, and finally to RTL implementation. The Python model is used as the floating-point golden reference and supports fast evaluation of different bit-widths and Q formats. The C++ fixed-point model then reproduces the integer behavior expected in hardware, which defines the bit-true semantics of each operator and serves as the reference for RTL verification.

As shown in Fig. 2, by comparing the MAE between the floating-point and C++ models, we can get wether the quantization strategy is effective; by comparing the C++ and RTL outputs, we can verify whether the hardware mapping is consistent with the software model. This process connects the numerical analysis and the accelerator datapath, making the fixed-point Slim-Mamba accelerator both hardware-efficient and numerically consistent with the deployment-oriented software model.

A. Quantization Workflow

Fig. 3 shows the proposed quantization workflow. It starts with a precision sweep to determine the required precision for

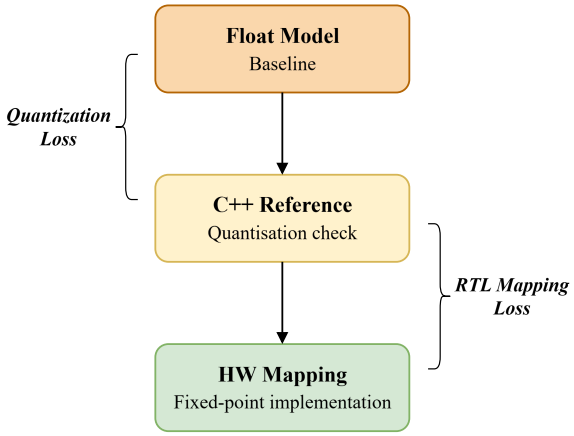

Fig. 2. The quantization tools: from floating-point evaluation to bit-true fixed-point modelling and RTL mapping.

different computation stages. A baseline error profiling step is then performed using a unified requantization configuration. If the final output error is too large, per-layer error analysis is used to locate where the mismatch grows. Based on this analysis, the layer-wise quantization strategy is refined and then mapped to the RTL quantization modules and memory initialization files.. A fake-quantization precision sweep is first inserted into the software model to estimate the accuracy impact of different quantization choices. In this step, quantize and dequantize operations are inserted in the software model to emulate low-precision effects, while the evaluation is still performed before RTL implementation. As shown in Table III, the all-int16 configuration achieves a localization error close to the fake-quantization reference. The sweep also shows that the `dt_proj`, `ssm_state`, and gate paths can be reduced to int8 with negligible loss, while `in_proj` and `out_proj` are more sensitive to precision reduction. However, the accelerator uses a shared reconfigurable MAC fabric for multiple projection stages, which will be discussed in Section IV. To keep the array control, datapath width, and on-chip memory organization simple, the final implementation adopts a unified 16-bit datapath instead of assigning different bit-widths to different stages. This choice slightly sacrifices the theoretical minimum bit-width, but it simplifies scheduling and allows the same MAC and requantization hardware to be reused across the Slim-Mamba block.

B. Layer-Wise Quantization Strategy

A unified fixed-scale baseline is first evaluated, where the main projection outputs and selective-scan state are represented in Q8.8 and use the same scale which is derived from the range of int16, $s = 2^{-8} = 1/256$. With this baseline, the hardware fixed-point output differs substantially from the floating-point reference, giving an output mean absolute error (MAE) of 0.17987 and a mean localization error of 0.35543 m. To locate the dominant error source, the Slim-Mamba block is decomposed into its main computation stages and the numerical mismatch is measured after each stage, as presented in Table IV.

TABLE III
FAKE-QUANTIZATION PRECISION SWEEP.

Configuration	Precision override	Mean localization error (m)
Fake-quant reference	Software fake-quant baseline	0.13695
All-int16	`ssm_state=int16, gate=int16`	0.13520
Mixed candidate	`dt_proj=int8, ssm_state=int8, gate=int8`	0.13519
`in_proj` int8	`in_proj=int8, ssm_state=int16, gate=int16`	0.76520
`out_proj` int8	`out_proj=int8, ssm_state=int16, gate=int16`	0.24352

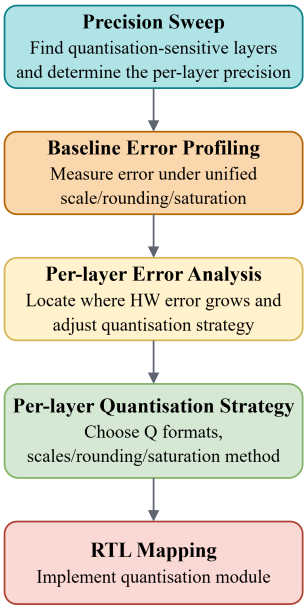

Fig. 3. Quantization workflow.

The stage-level analysis shows that the largest error increase occurs in the selective-scan state update. This is expected because the state is updated recurrently and its dynamic range varies across channels and time steps. A single fixed Q8.8 scale is therefore insufficient to represent both small and large state values. To reduce this error, the selective-scan state is changed to a dynamic-scale representation. The runtime scale is derived from the state range and used when storing and restoring the recurrent state. This reduces the selective-scan MAE from 0.127090 to 0.085484 and also improves the following gating and output-projection stages. The earlier stages remain unchanged because the dynamic scaling is introduced only at the selective-scan state path.

The final quantization strategy is summarized in Table V. Round-to-nearest-even (RNE) is used in the projection quantization and state-conversion paths because these paths contain repeated MAC accumulation and recurrent updates, where biased rounding errors can accumulate across layers. The sigmoid LUT is generated offline using nearest rounding (NR), while the runtime sigmoid path only performs clamping and table lookup. The sigmoid output `dt_lambda` is represented in Q0.16 and is converted to the Q1.15 state-update coefficient λ by a one-bit right shift. Since 1.0 is represented as 2^{15} in Q1.15, the complement $(1-\lambda)$ can be computed directly as $2^{15} - q_\lambda$, avoiding an extra quantization step.

TABLE IV
STAGE-LEVEL QUANTIZATION ERROR WITH DYNAMIC SCALING APPLIED TO THE STATE PATH.

Stage	Fixed-scale MAE	Dynamic-scale MAE
`norm`	0.004498	0.004498
`inproj_u`	0.010917	0.010917
`inproj_z`	0.011069	0.011069
`sigmoid_u`	0.006183	0.006183
`silu_gate`	0.005931	0.005931
`dtproj`	0.010139	0.010139
`dt_sigmoid`	0.002118	0.002118
`selective_scan`	**0.127090**	**0.085484**
`ewm_gating`	0.033001	0.029827
`outproj`	0.032225	0.031243

TABLE V
FINAL FIXED-POINT STRATEGY FOR THE SLIM-MAMBA ACCELERATOR.

Stage	Format	Scale	Rounding
RMSNorm output	Q8.8	2^{-8}	RNE
RMSNorm reciprocal	Q30	2^{-30}	NR
`in_proj`	Q8.8	2^{-8}	RNE
`dt_proj`	Q8.8	2^{-8}	RNE
`out_proj`	Q8.8	2^{-8}	RNE
Gate path	Q8.8	2^{-8}	RNE
Sigmoid LUT	Q0.16	2^{-16}	NR
`dt_lambda`	Q0.16	2^{-16}	NR
State coefficient λ	Q1.15	2^{-15}	shift
Selective-scan state	int16	dynamic	RNE

TABLE VI
END-TO-END QUANTIZATION EVALUATION.

Evaluation	Samples	HW vs. Float MAE	Mean Localization Error (m)
fixed scale	32	0.17987	0.35543
dynamic scale	34,505	0.02081	0.14103
Fake-quant ref.	34,505	–	0.13695

The improvement is also reflected in the final end-to-end evaluation in Table VI. The early fixed-scale baseline has a large output mismatch and a much higher localization error. With dynamic scaling, the end-to-end hardware fixed-point model achieves an output MAE of 0.02081 with respect to the floating-point output and a mean localization error of 0.14103 m, which is close to the fake-quantization reference of 0.13695 m, showing that the selected Q formats and dynamic state scaling preserve the localization accuracy while enabling fixed-point FPGA execution.

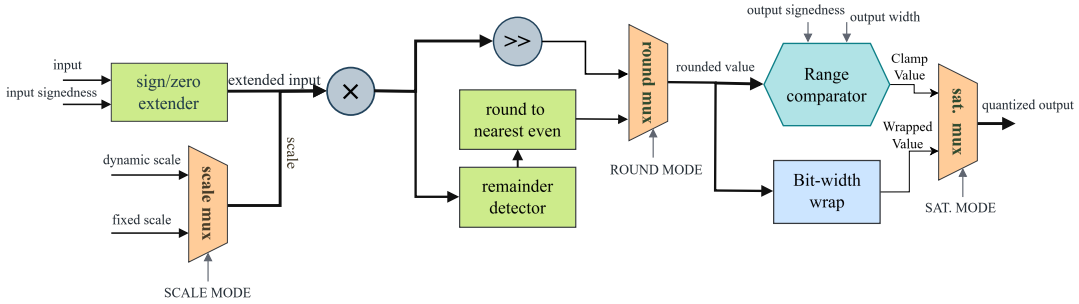

Fig. 4. Configurable fixed-point requantization engine reused across projection, state-update, and residual paths.

C. Configurable Requantization Engine

Instead of instantiating a separate quantization unit for each layer, the accelerator uses a shared configurable requantization engine. This module is used after MAC accumulation, Q-format conversion, state update, scale multiplication, and residual addition. As shown in Fig. 4, the input value is first sign-extended or zero-extended according to the input mode. It is then multiplied by either a fixed scale or a runtime dynamic scale. The product is shifted to the target binary point, rounded according to the selected rounding mode, and finally saturated or wrapped according to the output mode.

This shared structure reduces duplicated RTL and keeps the rounding and allows the layer-wise quantization policy obtained from software analysis to be adjusted easily to hardware control parameters. Different projection stages can therefore reuse the same datapath while selecting different scale memories, sign modes, and output formats.

The dynamic-scale state path requires additional alignment logic. Since the selective-scan datapath is stream based, each token, state address, update coefficient, and runtime scale must arrive at the state-update unit in the same order. A ping-pong scale buffer is used for this purpose, see Fig. 5. While one bank receives the incoming token and its generated runtime scale, the other bank provides an already aligned token-scale pair to the downstream state-update pipeline. The read and write banks switch between consecutive tokens, which hides the scale-generation latency and preserves streaming order. In this way, dynamic scaling can be inserted into the recurrent state path without breaking the fine-grained FIFO-based pipeline.

IV. Reconfigurable Slim-Mamba Hardware Design and Pipelining

This section presents the reconfigurable hardware architecture of the proposed Slim-Mamba accelerator. The term "reconfigurable" refers to:

- the same hardware chain can select different inter-block execution modes;
- the same MAC fabric can be scheduled for different Slim-Mamba projection operators;
- the tile shape and reduction length of the fabric can be configured for different matrix-vector shape with multi-mode PEs supporting addition, multiplication, and addition after multiplication;

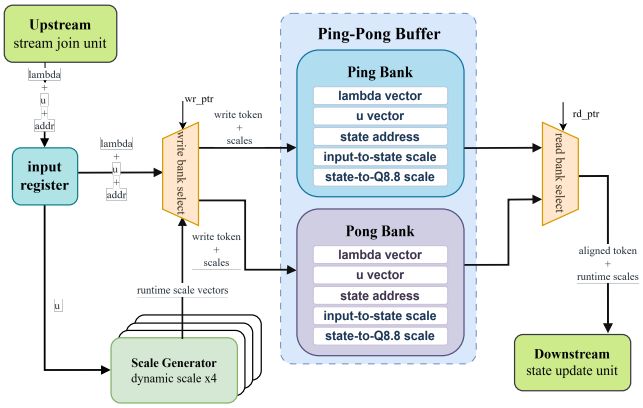

Fig. 5. Ping-pong scale-buffer organization for aligning token, state address, update coefficient, and runtime scale streams.

- the same fixed-point datapath can use different scale, rounding, and saturation policies for different layers and channels, see Fig. 4.

Fig. 6 shows the overall architecture. The design is implemented on a Zynq MPSoC platform. The processing system (PS) manages software control, DDR buffers, and AXI DMA transactions, while the programmable logic (PL) implements the Slim-Mamba compute datapath. As shown in Fig. 6(a), the contents of input buffer and output buffer are stored in PS-side DDR. The input vector is transferred to the PL through the memory-mapped-to-stream (MM2S) channel of the AXI DMA, and the output stream is written back to DDR through the stream-to-memory-mapped (S2MM) channel. The PS configures the accelerator using AXI-Lite control/status registers, including input preload, computation start, busy/status polling, and output write-back control.

A. Board-Level Dataflow and Four-Block Execution

The PL-side shell connects the AXI interfaces to the four-block Slim-Mamba compute chain, as shown in Fig. 6(b). The AXI-Lite CSR first starts the stream loader, which writes the incoming AXI-Stream data into the input buffer of Block 0. After the preload stage is complete, the four-block controller starts the block computation sequence. Each block contains a full set of operations in Slim-Mamba. Between adjacent blocks, the current block output is combined with the residual path using saturated fixed-point addition and then written into

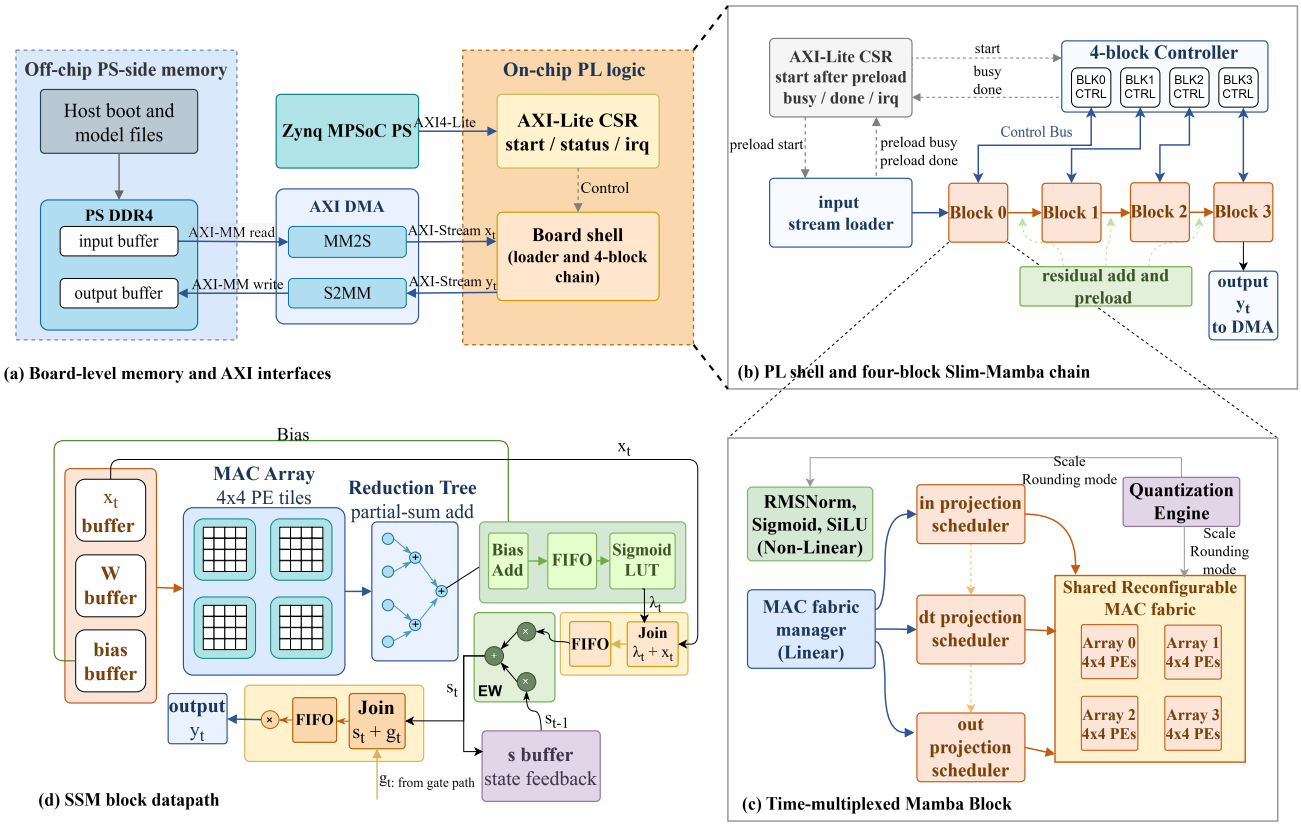

Fig. 6. Reconfigurable Slim-Mamba accelerator architecture: (a) board-level PS-side memory and AXI interfaces, (b) PL shell and four-block Slim-Mamba chain, (c) time-multiplexed Mamba block with shared projection fabric, and (d) SSM block datapath.

the next block input buffer. The output of the last block is streamed to the DMA output path.

The first reconfigurable feature appears at the block boundary. The four-block chain supports two inter-block transfer modes through the inter block pipeline control. When the pipeline is enabled, the accelerator uses a streaming inter-block transfer mode. In this mode, the output of Block (N) does not need to wait until the full block result has been cached. Instead, a registered inter-block write channel forwards the block output row by row or tile by tile into the input buffer of Block (N+1). Therefore, partial outputs from the current block can be made available to the next block earlier, improving the opportunity for overlap between block output generation and next-block preparation.

When the pipeline is unenabled, the accelerator uses a conservative preload mode. The output of the current block is first accumulated or cached completely. It is then preloaded into the input buffer of the next block, and the next block starts only after this preload step is complete. This mode has simpler control behavior and is useful for validation and debugging, while the pipeline mode is more throughput-oriented. Both modes use the same four-block hardware chain. The difference is the scheduling policy and the inter-block write behavior, rather than the physical compute units.

B. Reconfigurable Projection Fabric

Inside each Slim-Mamba block, the main MAC-intensive operators are the input projection, the update-gate projection in the SSM path, and the output projection. Although these operators use different input buffers, weight banks, and output destinations, they can all be represented as matrix-vector multiplication:

$$\mathbf{y} = W\mathbf{x} + \mathbf{b}. \tag{5}$$

The accelerator therefore avoids instantiating independent MAC arrays for these projection layers. Instead, it uses one shared $4 \times 4 \times 4$ MAC fabric, as illustrated in Fig. 6(c).

Each projection operator has its own scheduler. The in projection scheduler reads the block input or hidden representation and the input-projection weights, and converts them into activation and weight tiles. The dt projection scheduler reads the SSM input cache and the update-gate projection weights, and generates the corresponding tile stream. The out projection scheduler reads the gated SSM output and the output-projection weights. Although these three schedulers access different buffers and write to different destinations, they expose the same fabric interface. A MAC fabric manager arbitrates among the schedulers and configures the shared fabric to serve one projection role at a time.

This is the second reconfigurable feature of the accelerator. The physical MAC fabric remains unchanged, but its operator role is changed by scheduler selection, buffer selection, weight-bank selection, and output write-back configuration. In this way, the same MAC fabric is reused across the projection operators of the Slim-Mamba block. This reduces DSP duplication and avoids the routing and memory pressure that would arise from placing separate projection arrays in the same block.

The third reconfigurable feature is the configurable tile shape and reduction behavior of the same fabric. The fabric interface includes control parameters such as `mode`, `col_blocks_cfg`, and `reduce_rows`. The `mode` signal selects the current PE operation, such as MAC, EWM(element-wise multiplication), or EWA(element-wise addition). In the current accelerator, the shared fabric is mainly used in MAC mode for projection operators, while lightweight vector modules handle the fine-grained SSM element-wise path. However, the mode interface keeps the PE structure configurable and allows the same fabric abstraction to support different operator types.

The `col_blocks_cfg` parameter controls how many column blocks must be accumulated for one output tile. This is necessary because different projection layers have different input dimensions and therefore different reduction lengths along the K dimension. Rather than fixing the reduction length inside the PE array, each scheduler provides the required column-block count for its own projection workload. The `reduce_rows` signal controls whether the array outputs are reduced across rows to generate the final vector output. Together, these parameters allow the fixed $4 \times 4 \times 4$ physical array to support different logical matrix shapes, reduction lengths, and output forms.

The $4\times4\times4$ fabric contains four 4×4 sub-arrays. Each sub-array receives a four-element activation slice and a 4×4 weight block. The four sub-arrays operate with a staggered schedule and their partial results are combined through a reduction tree. Figure 7 illustrates the array organization for dt projection in SSM path. The B in the table stands for the current input, which is a 4×4 block. The weight matrix below is part of the weight mastrix for the dt projection, and each square means a 4×4 block. The four sub-arrays work in pipeline. At cycle 19, the first 4 rows of the weight matrix is completely consumed, and at cycle 22, the first output tile is produced after the reduction tree.

Because the main calculation in the projection operators is matrix-vector multiplication, compared with systolic array, our $4 \times 4 \times 4$ arrays organization reduces global routing and data-supply pressure for the MVM-dominated Slim-Mamba workload. Compared with a 64×1 GEMV engine, it consumes the same amount of hardware resources but provides earlier tile availability: still take SSM as example, which performs MVM with the matrix shape of (256,256) and vector shape of (256,1). With the four 4×4 sub-arrays working in pipeline, the first output vector can be produced after 22 cycles rather than waiting for a full vector-reduction interval in a 64×1 GEMV engine, which is 256 cycles. This earlier tile output is important because the following SSM nonlinear and state-update stages can start before the whole projection vector is completed.

C. Fine-Grained FIFO-Based Pipeline

As we memtioned before, the PEs inside the $4 \times 4 \times 4$ MAC arrays can be configured for MVM, EWM, EWA, so the trade-off between hardware reuse and latency needs to be considered. A coarse implementation of the SSM path would execute update-gate projection, sigmoid, state update, SiLU, and output gating sequentially. This maximizes resource reuse, but it forces each stage to wait until the previous stage has produced a full vector. The proposed accelerator instead uses a fine-grained FIFO-based pipeline. Once the MAC fabric produces and reduces an output tile, the tile is forwarded through FIFO interfaces to the bias-add, sigmoid, and state-update stages. The state-update unit starts as soon as the corresponding tile, update coefficient, state value, and scale are available. It does not wait for the full projection vector.

The state update follows the Slim-Mamba recurrence

$$h_t = \lambda_t \odot h_{t-1} + (1 - \lambda_t) \odot x_t^{(s)}, \tag{6}$$

where λ_t is generated by the update-gate projection and sigmoid unit. The updated state is then multiplied with the SiLU gate output. FIFOs are used to align the projection output, sigmoid value, gate path, state read result, and dynamic scale. This alignment is necessary because different sub-stages have different local latencies.

For an N-tile SSM workload, the coarse-grained schedule can be abstracted as

$$T_{\text{coarse}} = 29N, \tag{7}$$

where projection, nonlinear activation, state update, and output gating are mostly serialized. With fine-grained FIFO streaming, the schedule becomes

$$T_{\text{fine}} = 22N + 7. \tag{8}$$

For $N = 64$, this reduces the SSM-path latency from 1856 cycles to 1415 cycles, corresponding to a 23.8% reduction. The additional cost is about 12 DSPs per SSM compute path, which is acceptable compared with the latency reduction. Figure 8 shows the comparison between coarse-grained and fine-grained scheduling. Fig. 6(d) shows the final fine-grained SSM block datapath, where projection, bias addition, sigmoid, state update, SiLU gating, and output generation are connected through FIFO-based interfaces.

Overall, the accelerator combines four forms of architecture-level reconfigurability. The block chain can switch between streaming and preload execution. The shared MAC fabric can be configured by schedulers to execute different projection operators. The tile and reduction behavior of the fabric can adapt to different projection shapes. The quantization path can select different scale, rounding, and saturation policies for different layers and channels. These mechanisms allow the Slim-Mamba accelerator to reuse most of its hardware resources while preserving a fine-grained streaming path through the latency-sensitive SSM computation.

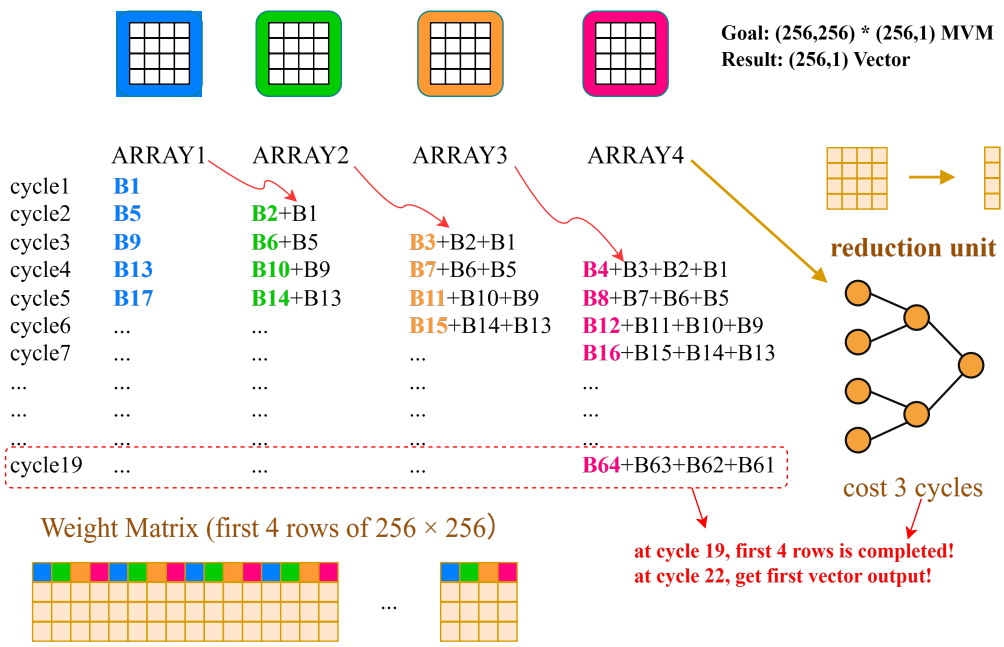

Fig. 7. $4 \times 4 \times 4$ arrays organization.

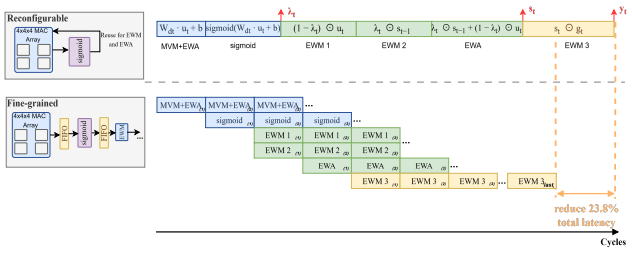

Fig. 8. Comparison between coarse-grained and fine-grained scheduling.

D. Nonlinear Layer Implementation

From the algorithm analyse in section II, the main nonlinear operators insede our Slim-Mamba model are RMSNorm, sigmoid, and SiLU.

RMSNorm is placed before the projection path to stabilize the dynamic range of the input features. For an input vector $\mathbf{x} = [x_0, x_1, \ldots, x_{D-1}]$, the RMS value is

$$\mathrm{RMS}(\mathbf{x}) = \sqrt{\frac{1}{D}\sum_{i=0}^{D-1} x_i^2 + \epsilon}, \tag{9}$$

and the normalized output is computed as

$$\hat{x}_i = x_i \cdot \frac{\widehat{1}}{\mathrm{RMS}(\mathbf{x})}. \tag{10}$$

The square-root operation is implemented with a trial-quotient digit-recurrence procedure, which uses shift, comparison, and subtraction operations instead of a general floating-point square-root unit, as shown in Figure 9. The final division is replaced by reciprocal generation followed by multiplication. A Q30 reciprocal is used so that the shared normalization scale remains accurate for all channels of one token.

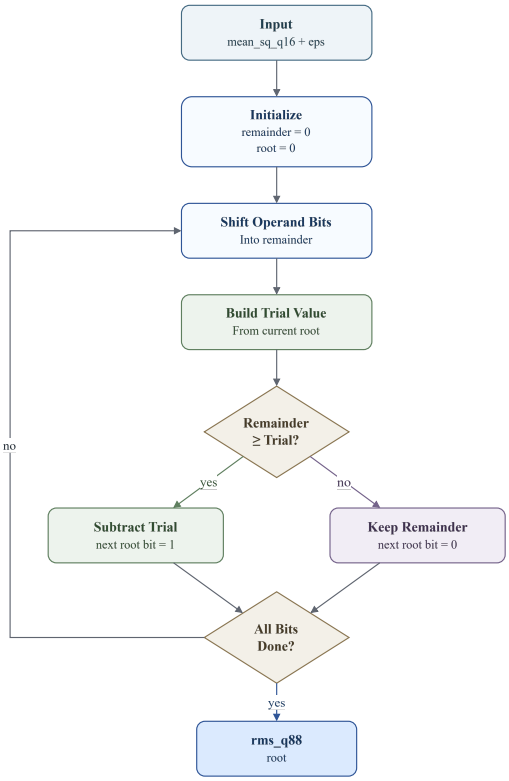

Fig. 9. RMSNorm approximation flow: a trial-quotient digit-recurrence procedure.

The sigmoid function is used to generate the state-update coefficient. It is implemented with a lookup table:

$$\sigma(x) = \frac{1}{1 + e^{-x}}. \tag{11}$$

The input is clamped to $[-4, +4)$, and a 2048-entry LUT

TABLE VII
SOFTWARE-SIDE MODEL COMPARISON.

Model	Input features	Size (MB)	Time (s/epoch)	it/s	Err. (cm)	HW cost proxy
FCNN [6]	CIR	789	N/A	N/A	~ 13	High
channel mamba	CIR + Power + amplitude	40.4	73	13.18	12.9	High
Vanilla Mamba v1	CIR + Power	3.38	33	32.62	24.74	Medium
Slim-Mamba	CIR + Power	3.40	13	75.30	13.5	Low

stores the corresponding Q0.16 sigmoid values. This avoids evaluating the exponential function in hardware. SiLU reuses the same sigmoid path:

$$\mathrm{SiLU}(x) = x \cdot \sigma(x). \tag{12}$$

The original input is delayed through a FIFO while the sigmoid value is produced. After alignment, an element-wise multiplier generates the SiLU output. Therefore, sigmoid and SiLU share the clamp and LUT logic, while SiLU only adds FIFO alignment and vector multiplication.

V. RESULTS

A. Software-Side Model Selection

Before presenting the FPGA measurements, we first justify the selection of Slim-Mamba as the deployment target. Table VII summarizes the main software-side model variants explored on LuViRA. The FCNN baseline provides a strong localization reference but at very large model size. Vanilla Mamba v1 is much more compact, but its localization accuracy is substantially worse on this task. The channel-aware Mamba variant achieves the best software-only mean error when augmented input features are used, but at the cost of substantially larger model size, longer training time, and a more complex multi-branch hardware mapping.

The best channel mamba result is not a strictly input-matched comparison against Slim-Mamba because it uses additional amplitude information. Figure 10 therefore compares Slim-Mamba with two channel mamba runs: one with the same $\mathrm{CIR} + \mathrm{Power}$ input as Slim-Mamba and one with the augmented input setting. Under matched input conditions, Slim-Mamba consistently outperforms channel mamba over most of the plotted range. This makes Slim-Mamba the more suitable deployment-oriented model, as it combines competitive localization accuracy with a much simpler operator structure.

The temporal window length was also studied because it directly affects both algorithmic context and hardware buffering cost. Table VIII shows that short windows are already sufficient for this task. In particular, $K = 2$ yields the same test mean error as $K = 1$ while providing a slightly lower evaluation error. Longer windows do not improve the final test accuracy and may even degrade it, despite occasionally reducing the evaluation error. This supports the use of a short deployed sequence window.

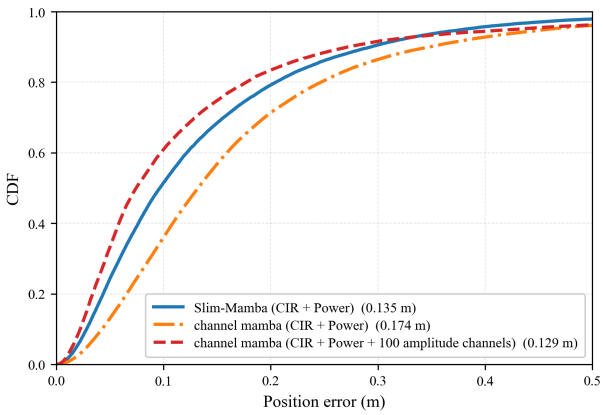

Fig. 10. Localization-error CDF for Slim-Mamba and two channel mamba reference runs under matched and augmented input settings.

TABLE VIII
ABLATION ON TEMPORAL WINDOW LENGTH K WITH PATCH LENGTH AND STRIDE VARIANTS.

K (patch length, stride)	Eval mean err. (m)	Test mean err. (m)
1 (1,1)	0.10068	**0.135**
2 (2,2)	0.09557	**0.135**
4 (2,2)	0.09814	0.139
8 (2,2)	0.09405	0.141
4 (4,4)	0.09983	0.137
6 (4,4)	0.09839	0.138
8 (4,4)	0.09749	0.144
8 (8,4)	0.09744	0.152
12 (8,4)	0.09286	0.156
16 (8,4)	0.09225	0.161
24 (8,4)	0.09432	0.159
32 (8,4)	0.09328	0.166

B. Quantization Accuracy

The second part of the evaluation concerns numerical consistency after fixed-point mapping. As summarized in Table VI, the early fixed-scale baseline produces a large output mismatch and a mean localization error of 0.35543 m. After introducing the final layer-wise quantization strategy and dynamic scaling for the recurrent state path, the hardware fixed-point model reaches an output MAE of 0.02081 with respect to the floating-point reference and a mean localization error of 0.14103 m. This is close to the software fake-quantization reference of 0.13695 m.

These results indicate that the main challenge is not fixed-point arithmetic itself, but the dynamic range of the recurrent state path. Once that path is stabilized by dynamic scaling,

TABLE IX
THROUGHPUT SUMMARY AT 100 MHZ.

Metric	Value
Target clock frequency	100 MHz
Required throughput	100 frames/s
Achieved throughput	471 frames/s
Average time per frame	2.12 ms
Requirement time per frame	10 ms

TABLE X
POST-SYNTHESIS RESOURCE UTILIZATION AT 100 MHZ.

Resource	Used	Available	Utilization
LUT	60868	274080	22.21%
FF	77345	548160	14.11%
BRAM tiles	279	912	30.59%
DSP	496	2520	19.68%

TABLE XI
ON-CHIP POWER ESTIMATE.

Metric	Value	Share
Total on-chip power	4.292 W	100%
Dynamic power	3.562 W	83%
Static power	0.729 W	17%
PS dynamic power	2.622 W	74% of dynamic power
PL dynamic power	0.940 W	26% of dynamic power

the hardware-oriented numerical behavior remains close to the floating-point model while using a unified 16-bit datapath in the main accelerator.

C. Throughput, Resource Utilization, and Power

Table IX summarizes the main throughput result. At 100 MHz, the accelerator reaches 471 frames/s, or about $4.7\times$ the 100 frames/s real-time target. This corresponds to an average processing time of 2.12 ms per frame.

Table X reports the post-synthesis resource usage. The design occupies 22.21% of LUTs, 14.11% of flip-flops, 30.59% of BRAM tiles, and 19.68% of DSPs. These numbers indicate that the shared MAC fabric and time-multiplexed block organization keep the implementation well within the device budget while supporting end-to-end deployment.

The power breakdown is shown in Table XI. The total on-chip power is 4.292 W, of which 3.562 W is dynamic and 0.729 W is static. The programmable logic accounts for about 0.94 W of dynamic power, while most of the dynamic component is contributed by the processing system and data movement. This indicates that further system-level optimization should target memory traffic and platform overhead in addition to arithmetic efficiency.

D. Latency Reduction from Fine-Grained Pipelining

The final set of results concerns the FIFO-based fine-grained pipeline introduced in the SSM path. As analyzed in Section IV, the coarse schedule can be approximated by $T_{\mathrm{coarse}} = 29N$, whereas the fine-grained schedule becomes $T_{\mathrm{fine}} = 22N + 7$. For $N = 64$ tiles, this reduces the SSM-path latency from 1856 cycles to 1415 cycles, corresponding to a 23.8% reduction.

TABLE XII
LATENCY BREAKDOWN PER SLIM-MAMBA BLOCK.

Stage	Cycles	Time at 100 MHz
Input projection	2292	22.92 μs
SSM path	1415	14.15 μs
Output projection	1093	10.93 μs
Control / alignment overhead	501	5.01 μs
Total per block	**5301**	**53.01 μs**

The per-block latency breakdown is reported in Table XII. One Slim-Mamba block requires 5301 cycles, or 53.01 μs at 100 MHz. Input projection remains the dominant stage, but the latency-sensitive state path is no longer the main bottleneck. This is the intended effect of the fine-grained pipeline: the recurrence and nonlinear stages start on partial tiles instead of waiting for a full projection vector.

Overall, the results support the main claim of this work. Slim-Mamba is not only a compact Mamba-like localization model, but also a more practical hardware target than the heavier software-oriented alternatives explored during model selection. The algorithmic simplification, layer-wise quantization strategy, and reconfigurable hardware architecture together produce an FPGA implementation that preserves localization accuracy, exceeds the real-time requirement with margin, and uses moderate resources on the target Zynq MPSoC platform.

VI. CONCLUSIONS

This paper presented Slim-Mamba, a compact Mamba-based localization model and its reconfigurable FPGA accelerator for Massive MIMO indoor localization. The proposed approach combines task-driven model simplification, layer-wise fixed-point quantization, and hardware architecture co-design. Relative to the original Mamba state path, Slim-Mamba retains recurrent temporal modeling and input-dependent gating while removing the discretization and selective-scan operators that are less suitable for the target deployment setting. The resulting accelerator achieves a mean localization error of 0.14103 m, reaches 471 frames/s at 100 MHz, and uses moderate FPGA resources on the target Zynq MPSoC platform. These results show that Slim-Mamba provides an effective deployment-oriented tradeoff among localization accuracy, numerical efficiency, and hardware cost.

REFERENCES

[1] S. E. Trevlakis, A.-A. A. Boulogeorgos, D. Pliatsios, J. Querol, K. Ntontin, P. Sarigiannidis, S. Chatzinotas, and M. Di Renzo, "Localization as a key enabler of 6g wireless systems: A comprehensive survey and an outlook," *IEEE Open Journal of the Communications Society*, vol. 4, pp. 2733–2801, 2023.

[2] F. Zafari, A. Gkelias, and K. K. Leung, "A survey of indoor localization systems and technologies," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2568–2599, 2019.

[3] I. Yaman, G. Tian, M. Larsson, P. Persson, M. Sandra, A. D"urr, E. Tegler, N. Challa, H. Garde, F. Tufvesson, K. Åstr"om, O. Edfors, S. Malkowsky, and L. Liu, "The LuViRA dataset: Synchronized vision, radio, and audio sensors for indoor localization," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 11 920–11 926.

[4] I. Yaman, G. Tian, E. Tegler, J. Gulin, N. Challa, F. Tufvesson, O. Edfors, K. Åstr"om, S. Malkowsky, and L. Liu, "LuViRA dataset validation and discussion: Comparing vision, radio, and audio sensors for indoor localization," *IEEE Journal of Indoor and Seamless Positioning and Navigation*, vol. 2, pp. 240–250, 2024.

[5] S. Malkowsky, J. Vieira, L. Liu, P. Harris, K. Nieman, N. Kundargi, I. C. Wong, F. Tufvesson, V. "Owall, and O. Edfors, "The world's first real-time testbed for massive MIMO: Design, implementation, and validation," *IEEE Access*, vol. 5, pp. 9073–9088, 2017.

[6] G. Tian, I. Yaman, M. Sandra, X. Cai, L. Liu, and F. Tufvesson, "High-precision machine-learning based indoor localization with massive MIMO system," in *ICC 2023 – IEEE International Conference on Communications*, Rome, Italy, 2023, pp. 3690–3695.

[7] ——, "Deep-learning-based high-precision localization with massive MIMO," *IEEE Transactions on Machine Learning in Communications and Networking*, vol. 2, pp. 19–33, 2024.

[8] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.

[9] R. Wei, S. Xu, L. Zhong, Z. Yang, Q. Guo, Y. Wang, R. Wang, and M. Li, "LightMamba: Efficient mamba acceleration on FPGA with quantization and hardware co-design," in *2025 Design, Automation & Test in Europe Conference (DATE)*, 2025.

[10] A. Wang, H. Shao, S. Ma, and Z. Wang, "Fastmamba: A high-speed and efficient mamba accelerator on fpga with accurate quantization," in *2025 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, vol. 1, 2025, pp. 1–6.

[11] X. Jin, H. Zheng, M. Nie, J. Wang, and C. R. Shi, "HCSAs: Hybrid computing systolic arrays for accelerating mamba models with unified state space buffers and energy-efficient dataflow," in *Proceedings of the Great Lakes Symposium on VLSI 2025*. New York, NY, USA: ACM, 2025, pp. 457–462.